COOLPEOPLE · FRONTEND

Building consistent, strong frontend

A short checklist of habits that separate ad-hoc CSS from a design system. Every rule here traces back to one idea: **decide a value once, reference it everywhere.**

THE ONE PRINCIPLE

Hardcoding a color or a pixel value is a decision you'll have to remember and repeat. A **token** is that decision, named once. When the brand gold changes, you edit one line instead of hunting through forty files. Consistency isn't willpower — it's not having the option to be inconsistent.

Rule of thumb: if you type a hex color or a raw px spacing value anywhere outside `:root`, stop — there should be a `var(--…)` for it. If there isn't, add the token first, then use it.

THE HABITS

1 Initialize design tokens in `:root`, once, globally

Put colors, spacing, radii, shadows, and motion in one global stylesheet (imported at your app entry). Reference them everywhere with `var()`.

```
:root{
  --ink:#111114; --ink-2:#3d3d44; --muted:#8b8b93; --line:#e8e8ea;
  --paper:#fff; --gold:#c9a437; --gold-deep:#a9861d; --gold-soft:#f6eccf;
  --s2:8px; --s3:12px; --s4:16px; --s5:24px; --s6:32px; --s8:48px;
  --radius:18px;
  --shadow:0 1px 2px rgba(16,16,20,.04), 0 12px 34px rgba(16,16,20,.06);
  --ease:.18s ease;
}
```

2 Never hardcode a value that has a token

The whole point of tokens is defeated the moment you paste `#a9861d` into a component.

AVOID

```
.number{ color:#8b8b93; }
.image{ background:#f6eccf;
        padding:10px;
        border-radius:12px; }
```

PREFER

```
.number{ color:var(--muted); }
.image{ background:var(--gold-soft);
        padding:var(--s2);
        border-radius:var(--radius-sm); }
```

3 Use a spacing scale — not arbitrary numbers

`10px` here, `13px` there, `25px` somewhere else reads as visual noise. Pick a scale (8·12·16·24·32·48) and snap to it. Rhythm looks intentional; randomness looks broken.

4 In hover / state rules, override *only* what changes

Repeating every property in `:hover` is a bug waiting to happen — change the base and the hover silently drifts out of sync.

AVOID — REPEATS EVERYTHING

```
.card:hover .image{
  background:var(--gold);
  color:#000;
  padding:10px;        /* dup */
  border-radius:12px;  /* dup */
  max-height:50px;     /* dup */
}
```

PREFER — ONLY THE DELTA

```
.card:hover .image{
  background:var(--gold);
  color:var(--gold-ink);
}
```

5 Prefer CSS `:hover` over JS state for visual-only changes

No `useState`, no handlers, no re-renders, and it's automatically scoped per element. Reserve state-driven hover for things CSS can't do (fetching a preview, opening a tooltip component).

```
/* scopes to each card for free */
.miniContainer:hover .image { background:var(--gold); color:var(--gold-ink); }
.miniContainer:hover .number { color:var(--gold); }
```

6 Animate transitions on the properties that change

A hover that snaps feels cheap. Add a `transition` on the base element (not the `:hover`) so it eases both in and out.

```
.miniContainer{ transition:transform var(--ease), box-shadow var(--ease); }
.miniContainer:hover{ transform:translateY(-3px); box-shadow:var(--shadow-hover); }
```

7 Equal columns → CSS Grid, not flex hacks

For N equal cards, `grid-template-columns:repeat(4,1fr)` gives perfectly even columns and trivial responsive breakpoints. Flex + `min-width` fights you.

```
.grid{ display:grid; grid-template-columns:repeat(4,1fr); gap:var(--s5); }
@media(max-width:900px){ .grid{ grid-template-columns:repeat(2,1fr); } }
@media(max-width:560px){ .grid{ grid-template-columns:1fr; } }
```

8 Inline SVG + `currentColor` for tintable icons

An `<img src="icon.svg">` can't be recolored with CSS. Inline the SVG and set its `stroke` / `fill` to `currentColor` — now one `color:` rule tints it, including on hover.

```
<svg viewBox="0 0 24 24" fill="none" stroke="currentColor">…</svg>
.badge{ color:var(--gold-deep); }         /* icon color */
.card:hover .badge{ color:var(--gold-ink); }
```

9 Reset `box-sizing` once, globally

`*{box-sizing:border-box}` makes `width` include padding & border — so layouts add up the way you expect. Set it once in the global sheet and never think about it again.

10 Map over data instead of copy-pasting markup

Four near-identical cards = one array + one `.map`. No divergent copies to keep in sync; adding a fifth is one data entry.

```
const STEPS = [{num:'01', title:'…', body:'…', paths:[…]}, …];
STEPS.map((s,i) => <Card key={i} {...s} />)
```

WORKED EXAMPLE — THE "HOW IT WORKS" CARDS

Same component, before and after applying the habits above.

BEFORE

```
.image{
  background:#f6eccf; color:#a9861d;
  padding:10px; border-radius:12px;
  margin-top:20px; max-height:50px;
}
.card:hover .image{
  background:#a9861d; color:black;
  padding:10px; border-radius:12px;   /* dup */
  margin-top:20px; max-height:50px;   /* dup */
}
/* hardcoded, repeated, no transition */
```

AFTER

```
.image{
  width:24px; height:24px;
  background:var(--gold-soft);
  color:var(--gold-deep);
  padding:var(--s2);
  border-radius:var(--radius-sm);
  transition:background var(--ease),
             color var(--ease);
}
.card:hover .image{
  background:var(--gold);
  color:var(--gold-ink);
}
```

The 30-second self-review before you commit CSS: ① any raw hex or px that should be a token? ② does the `:hover` repeat properties it shouldn't? ③ is there a `transition`? ④ could this markup be a `.map`?

Coolpeople frontend guide · reference implementation lives in `src/index.css` (tokens) and `HowItWorksTimeline.css` (usage).